

從筆記本到 Kubeflow Pipelines (含超參數調整功能)：資料科學之旅

來源：<https://codelabs.developers.google.com/codelabs/cloud-kubeflow-minikf-kale-katib?hl=zh-tw>

步驟數：7

在這個程式碼研究室中，您將瞭解如何在不使用任何 CLI 指令或 SDK 的情況下，在 Kubeflow 管道上透過超參數調整建構及部署複雜的數據資料學管道。

目錄

1. [1. 簡介](#)
2. [2. 設定環境](#)
3. [3. 安裝 MiniKF](#)
4. [4. 從筆記本內執行管道](#)
5. [5. 使用超參數調整進行遷移學習](#)
6. [6. 清除所用資源](#)
7. [7. 恭喜](#)

1. 簡介



[Kubeflow](#) 是 [Kubernetes](#) 的機器學習工具包，這項專案旨在簡化在 Kubernetes 中部署機器學習 (ML) 工作流程的作業，並提高其可攜性與可擴充性。此專案目標是提供直觀的方式，協助使用者將領先業界的 **ML** 專屬開放原始碼系統部署至各個不同的基礎架構。

Kubeflow 部署作業的流程為何？

Kubeflow 部署作業是指：

- 可攜性 - 無論 Kubernetes 叢集位於 Google Cloud Platform (GCP)、內部部署環境或不同供應商之間，都可以使用。
- 可擴充：可使用波動的資源，且只受限於分配給 Kubernetes 叢集的資源數量。
- 可組合 - 您可以從精選的機器學習架構和程式庫中選擇，將獨立步驟設定為完整的機器學習工作流程。

Kubeflow 可讓您將鬆散耦合的微服務整理為單一單元，並部署至各種位置，包括筆電、內部部署或雲端。

本程式碼研究室將逐步說明如何使用 MiniKF 建立自己的 Kubeflow 部署作業，然後執行 Kubeflow Pipelines 工作流程並進行超參數調整，以訓練及提供模型。您可以在 Jupyter Notebook 中完成所有操作。

想瞭解更多資訊嗎？請參閱 [Kubeflow Examples 存放區](#)，瞭解最新逐步說明。

建構項目

在本程式碼研究室中，您將在 Kubeflow Pipelines 上建構複雜的資料科學管道，並進行超參數調整，期間不需使用任何 CLI 指令或 SDK。您不需要具備任何 Kubernetes 或 Docker 知識，完成後，基礎架構會包含：

- MiniKF (Mini Kubeflow) VM，會自動安裝下列項目：
- Kubernetes (使用 Minikube)
- [Kubeflow](#)
- [Kale](#)：可將一般用途的 Jupyter Notebook 轉換為 Kubeflow Pipelines 工作流程 ([GitHub](#))
- [Arrikto Rok](#)，用於資料版本控管和可重現性

課程內容

- 如何使用 MiniKF 安裝 Kubeflow

- 如何將 Jupyter 筆記本轉換為 Kubeflow Pipelines，而不使用任何 CLI 指令或 SDK
- 如何從筆記本中執行 Kubeflow Pipelines，並透過超參數調整功能，只要按一下按鈕即可
- 如何在筆記本和每個管道步驟中自動為資料建立版本

軟硬體需求

- 您擁有「擁有者」權限的有效 [GCP 專案](#)

本程式碼研究室為進階主題，著重於 Kubeflow。如需更多背景資訊和平台簡介，請參閱「[Kubeflow 簡介](#)」說明文件。我們不會對與主題無關的概念和程式碼多做介紹，但會事先準備好這些程式碼區塊，屆時您只要複製及貼上即可。

2. 設定環境

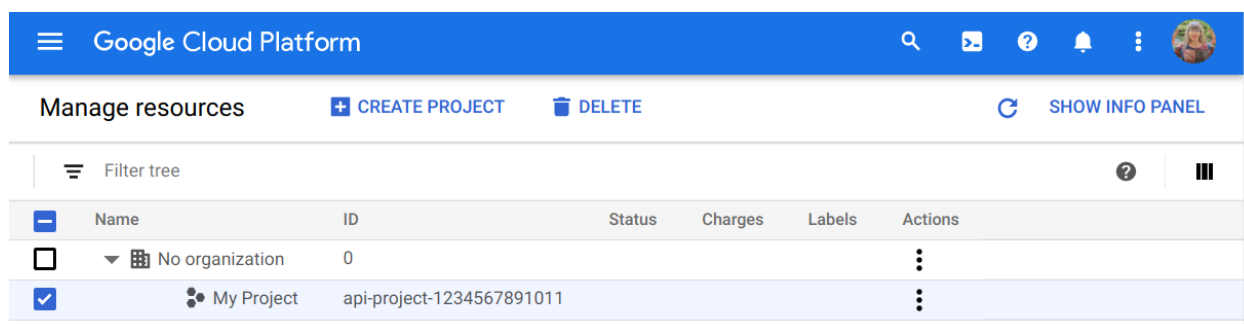
設定 GCP 專案

如要使用自己的 GCP 專案，請按照下列步驟操作

如果您已獲得專為本程式碼研究室預先分配的 GCP 專案，可以略過本節。直接前往下一節，開啟專案。

請按照下列步驟建立 GCP 專案，或設定現有的 GCP 專案。如果您打算使用現有 GCP 專案，請確認專案符合下列最低需求。第一步是在 GCP 主控台中開啟資源管理工具。

建立新專案或選取現有專案：



請確認符合下列最低需求：

- 確認您具備專案的**擁有者角色**。
- 確認專案已啟用「帳單」。
- 如果您使用 [GCP 免費方案](#)，或享有 12 個月的試用期和 \$300 美元抵免額，請注意，您無法執行 MiniKF 的預設 GCP 安裝作業，因為免費方案提供的資源不足。您需要**升級為付費帳戶**。

如需設定 GCP 專案的詳細說明，請參閱 [GCP 說明文件](#)。

設定 GCP 專案後，請直接參閱 MiniKF 安裝說明。

開啟預先分配的 GCP 專案

如果您有預先分配的 GCP 專案，請按照下列步驟操作

如果您使用自己的 GCP 專案，則可略過此部分。如要安裝 MiniKF，請直接前往下一節。

如要開啟預先分配的 GCP 專案，請按一下下方按鈕前往 GCP 主控台，然後開啟左上角漢堡選單中的「首頁」面板。如果畫面空白，請在提示中按一下「是」建立資訊主頁。

The screenshot shows the Google Cloud Platform interface. The top navigation bar includes the Google Cloud Platform logo, a dropdown menu currently showing 'Kubeflow Pipelines 5005', and a search icon. On the left sidebar, the 'Home' link is circled in red. The main content area is divided into 'DASHBOARD' and 'ACTIVITY' tabs. Under the 'DASHBOARD' tab, there is a 'Project info' section. This section displays the project name 'Kubeflow Pipelines 5005', the 'Project ID' 'kckp19-5005' (which is circled in red), and the project number '668007489325'. Below this information is a button labeled 'ADD PEOPLE TO THIS PROJECT' and a link 'Go to project settings'. A 'Resources' section is also visible below the project info.

如果尚未選取專案，請按一下「選取專案」：

This screenshot shows the Google Cloud Platform interface with the 'Select a project' dropdown menu highlighted by a red circle in the top navigation bar. The left sidebar shows the 'Home' link. The main content area is titled 'Dashboard' and contains a message box with an information icon and the text 'To view this page, select a project.'

選取所需的專案，你應該只會看到一個：

Select a project



NEW PROJECT

Search projects and folders

Q |

RECENT ALL

Name		ID
Kubeflow Pipelines 5005 ?		kckp19-5005

CANCEL OPEN

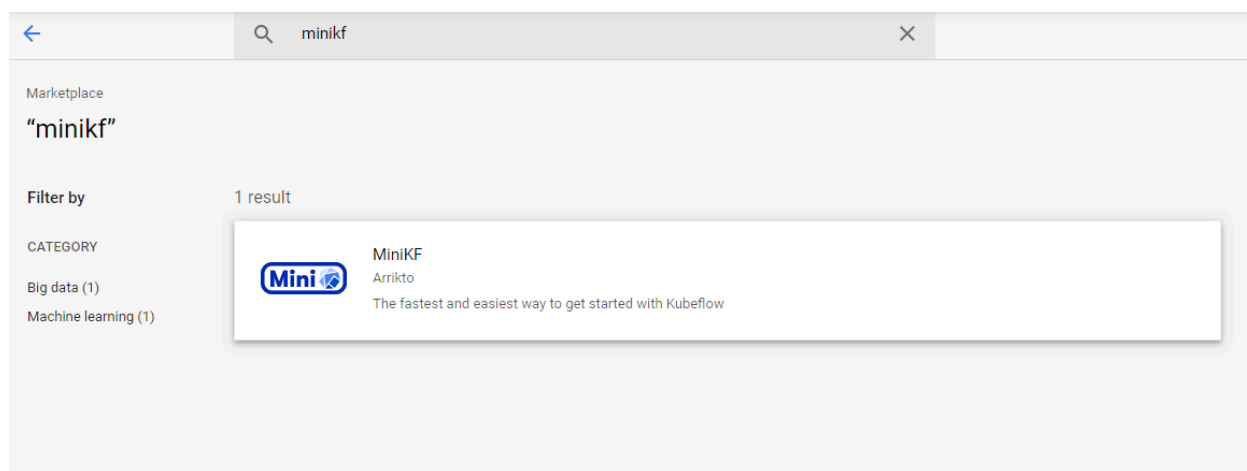
步驟 3 · 約 15 分鐘

3. 安裝 MiniKF

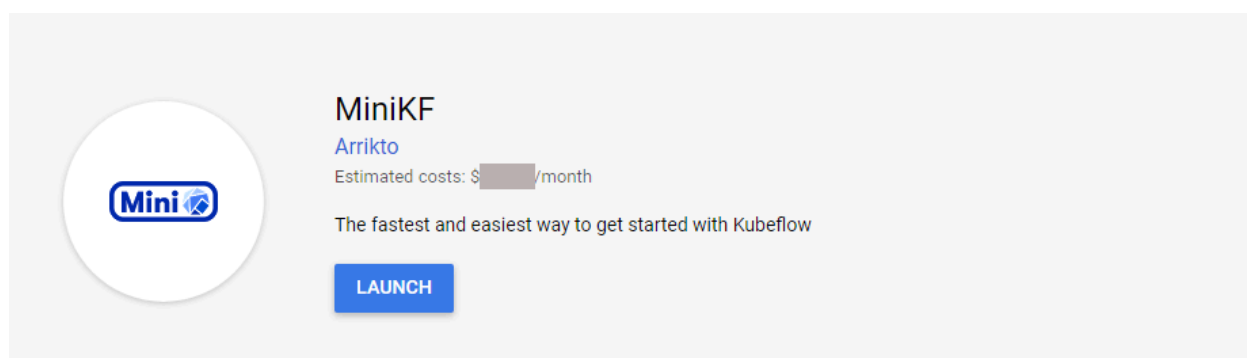
建立包含 MiniKF 的 Compute 執行個體

在 GCP Marketplace 中搜尋「MiniKF」。

選取 Arrikto 的 **MiniKF** 虛擬機器：



按一下「啟動」按鈕，然後選取專案：



Runs on
Google Compute Engine

Type
[Virtual machines](#)
Single VM

Last updated
[redacted], 6:56 AM

Category
[Big data](#)
[Machine learning](#)

Version
[redacted]

Operating system
Ubuntu Xenial 16.04.6 LTS

Overview

A production-ready, full-fledged Kubeflow deployment that installs in minutes.

MiniKF is the fastest and easiest way to get started with Kubeflow. With just a few clicks, you are up for experimentation, and for running complete Kubeflow Pipelines.

NEW! Now with Kubeflow 1.0 and a brand-new Volume Manager UI.

[Learn more](#) [↗](#)

About Arrikto

Arrikto creates software to transform how distributed applications discover and consume data on-prem or on the cloud.

[Learn more](#)

[About the provider](#) [↗](#)

在「Configure & Deploy」視窗中，為 MiniKF 執行個體選擇**名稱**和**區域**，並保留預設選項。然後按一下「部署」按鈕：

New MiniKF deployment

Deployment name

Zone

Machine type

30 GB memory

[Customize](#)

Boot Disk

Boot disk type

Boot disk size in GB

Data Disk

Data disk type

Data disk size in GB

GPUs

GPU type

The number of GPU dies is linked to the number of CPU cores and memory selected for this instance. For the current configuration, you can select no fewer than 1 GPU die of this type. [Learn more](#)

Number of GPUs

GPU type

 Machines with GPUs can't migrate on host maintenance

Networking

Network interfaces

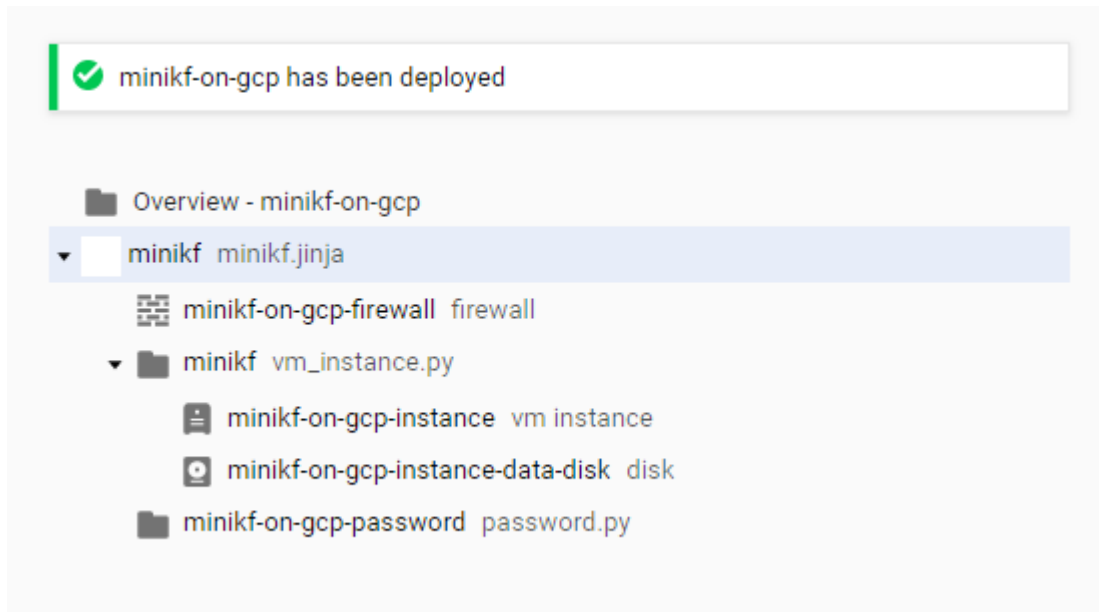
 Add network interface

 You have reached the maximum number of one network interface

 [More](#)

Deploy

等待 MiniKF Compute 執行個體啟動：



登入 MiniKF

MiniKF VM 啟動後，按一下「SSH」**SSH**按鈕即可連線並登入。按照畫面上的指示執行 `minikf` 指令，開始部署 Minikube、Kubeflow 和 Rok。這項作業需要幾分鐘才能完成。

Get started with MiniKF



Suggested next steps

- **Ensure MiniKF is up and running**
Before visiting the MiniKF dashboard, ensure that Minikube, Kubeflow, and Rok are up and running. To do so, click on the SSH button to connect to the MiniKF VM and follow the on-screen instructions.
- **Log in to Kubeflow**
Visit the MiniKF dashboard and log in to Kubeflow using the MiniKF username and password.
- **Log in to Rok**
After logging in to Kubeflow, navigate to the Snapshot Store and log in to Rok, using again the MiniKF username and password.

登入 Kubeflow

安裝完成且所有 Pod 都準備就緒後，請前往 MiniKF 資訊主頁。使用 MiniKF 使用者名稱和密碼登入 Kubeflow：

MiniKF

Solution provided by Arrikto Inc.

MiniKF dashboard	
MiniKF username	user
MiniKF password	Fan3LTt72b
Instance	minikf-on-gcp-instance
Instance zone	us-west2-a
Instance machine type	n1-standard-8

Log in to Your Account

Username

Password

Login

請注意，MiniKF 使用的是自行簽署的憑證，因此您必須點選「進階設定」，然後前往這個地址。

Chrome 使用者會看到這個畫面：



Your connection is not private

Attackers might be trying to steal your information from [REDACTED] (for example, passwords, messages, or credit cards). [Learn more](#)

NET::ERR_CERT_AUTHORITY_INVALID

Hide advanced

Back to safety

This server could not prove that it is [REDACTED]; its security certificate is not trusted by your computer's operating system. This may be caused by a misconfiguration or an attacker intercepting your connection.

[Proceed to \[REDACTED\] \(unsafe\)](#)

Firefox 使用者會看到這個畫面：



Warning: Potential Security Risk Ahead

Firefox detected a potential security threat and did not continue to [REDACTED]. If you visit this site, attackers could try to steal information like your passwords, emails, or credit card details.

[Learn more...](#)

Go Back (Recommended)

Advanced...

Websites prove their identity via certificates. Firefox does not trust this site because it uses a certificate that is not valid for [REDACTED]. The certificate is only valid for the following names: localhost, 10.10.10.10

Error code: [MOZILLA_PKIX_ERROR_SELF_SIGNED_CERT](#)

[View Certificate](#)

Go Back (Recommended)

Accept the Risk and Continue

Safari 使用者會看到這個畫面：



This Connection Is Not Private

This website may be impersonating [redacted] to steal your personal or financial information. You should go back to the previous page.

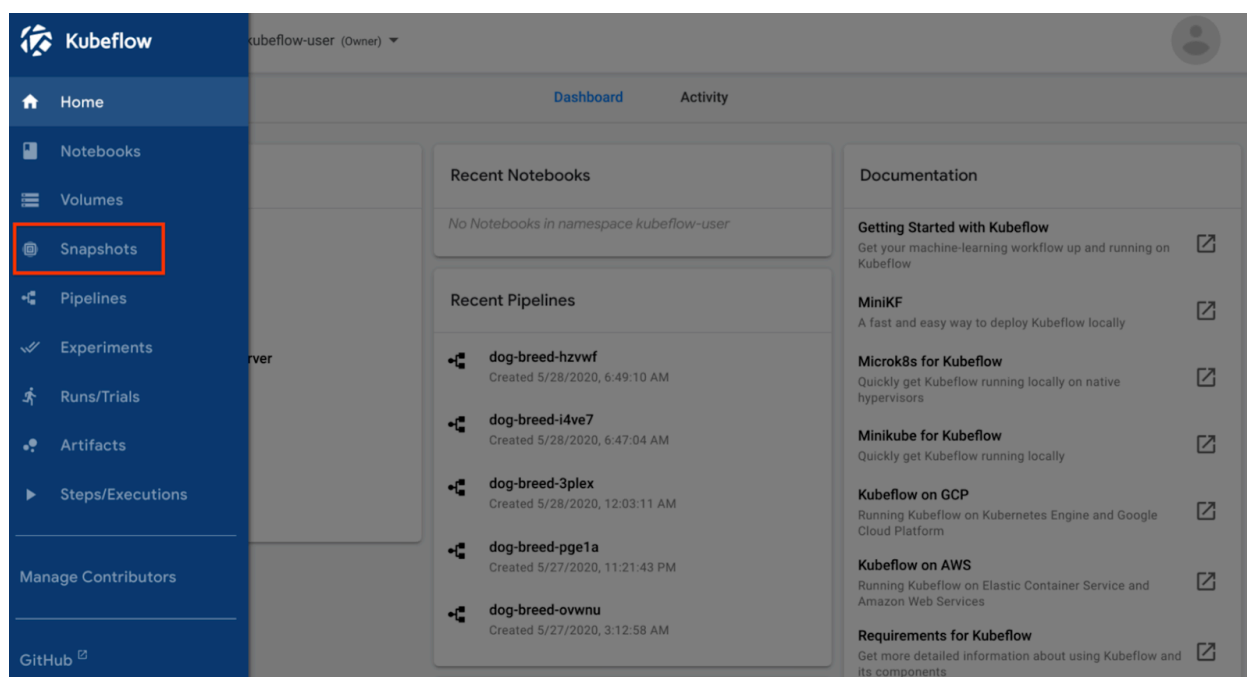
Go Back

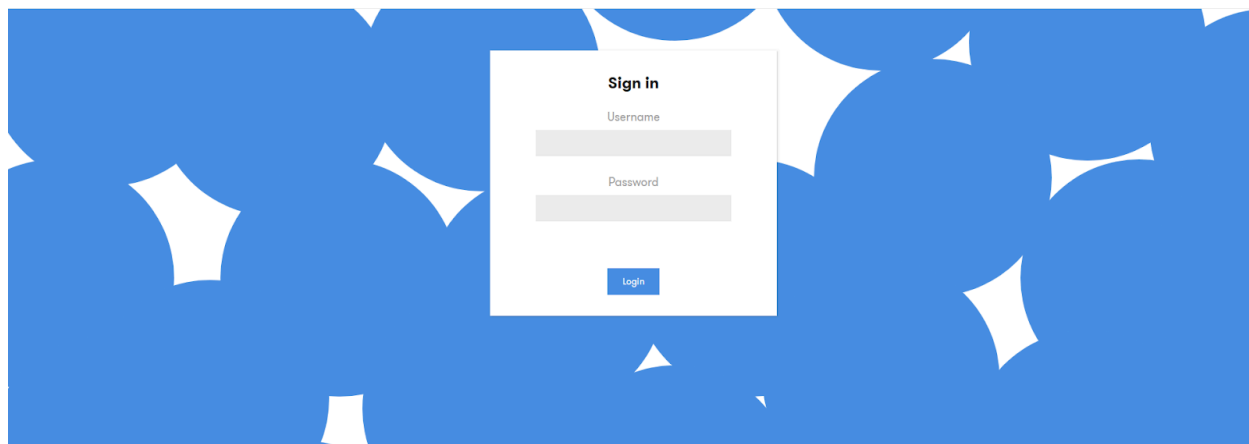
Safari warns you when a website has a certificate that is invalid. This may happen if the website is misconfigured or an attacker has compromised your connection.

To learn more, you can [view the certificate](#). If you understand the risks involved, you can [visit this website](#).

登入 Rok

登入 Kubeflow 後，按一下漢堡圖示開啟左選單。按一下「快照」，然後使用 MiniKF 使用者名稱和密碼登入 Rok。





恭喜！您已成功在 GCP 上部署 MiniKF。您現在可以建立筆記本、撰寫機器學習程式碼、執行 Kubeflow Pipelines，以及使用 Rok 進行資料版本控管和重現。

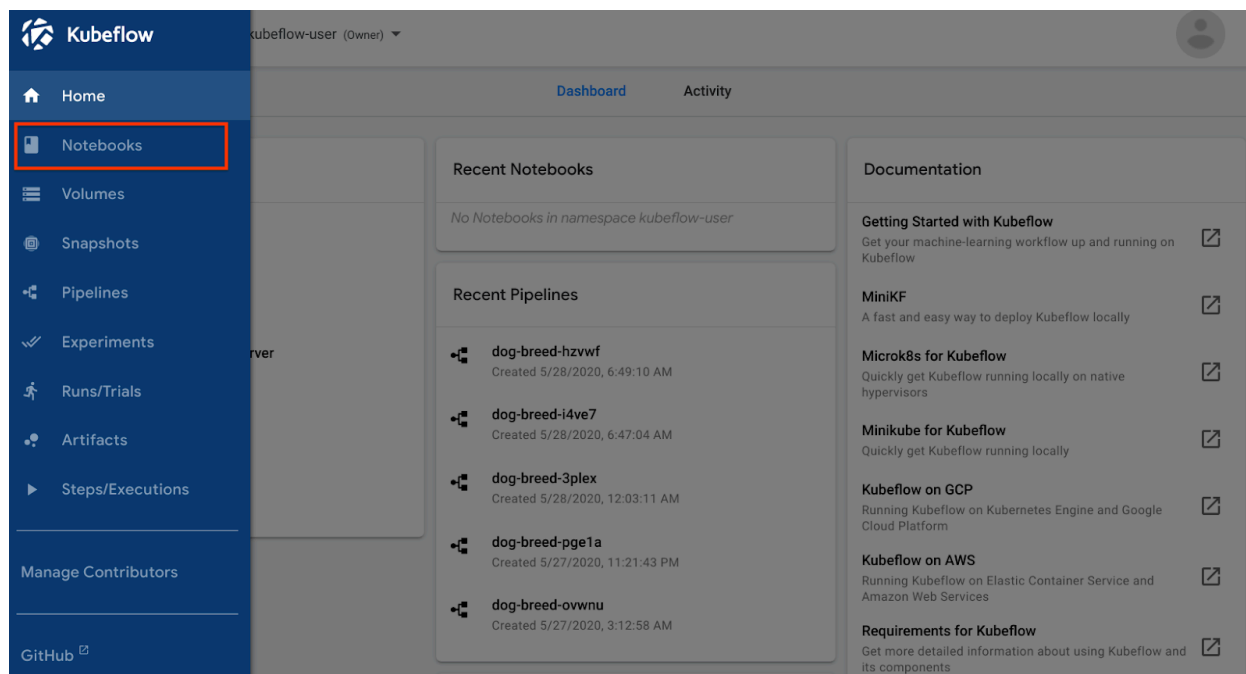
步驟 4 · 約 30 分鐘

4. 從筆記本內執行管道

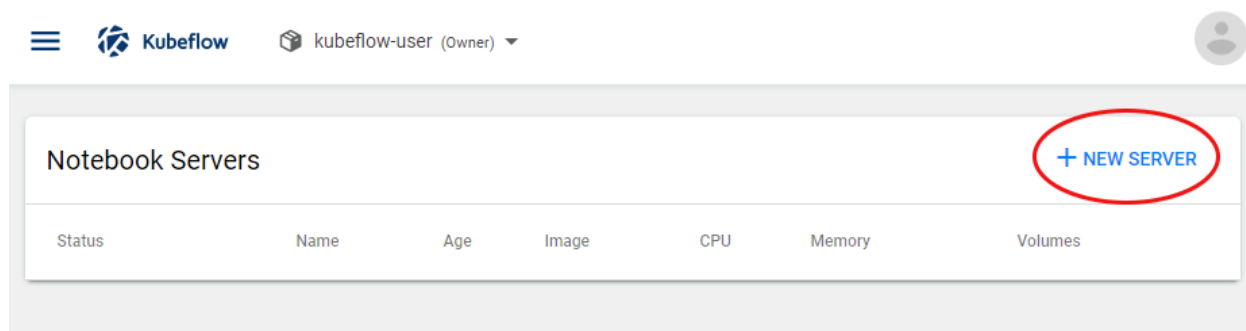
在本節中，您將執行 Udacity AI Nanodegree 的專案「Dog Breed Identification」範例。只要提供狗的圖片，最終模型就會估算狗的品種。

在 Kubeflow 叢集中建立筆記本伺服器

前往 Kubeflow 中央資訊主頁的「Notebooks」(筆記本) 連結。



按一下「新增伺服器」。



指定筆記本伺服器的名稱。

 **Rok JupyterLab URL**

Load an existing Jupyter Lab by providing a valid Rok URL.



AUTOFILL

 **Name**

Specify the name of the Notebook Server and the Namespace it will belong to.

kubecon-tutorial

kubeflow-user

 **Image**

A starter Jupyter Docker Image with a baseline deployment and typical ML packages.

請確認您已選取下列 Docker 映像檔 (請注意，映像檔標記可能有所不同)：

```
gcr.io/arrikto/jupyter-kale:f20978e
```

新增大小為 5 GB 的空白資料磁碟區，並命名為「data」。

Workspace Volume

Configure the Volume to be mounted as your personal Workspace.

☐ Don't use Persistent Storage for User's home

Type New	Rok URL	Name workspace-kubecon-tutor	Size 5Gi	Mount Point /home/jovyan
-------------	---------	---------------------------------	-------------	-----------------------------

Data Volumes

Configure the Volumes to be mounted as your Datasets.

+ ADD VOLUME

Type New	Rok URL	Name data	Size 5Gi	Mount Point /home/jovyan/data	
-------------	---------	--------------	-------------	----------------------------------	---

Configurations

按一下「啟動」即可建立筆記本伺服器。

⚙️ Extra Resources

Specify extra resources that might be needed in the Notebook Server.

☒ Enable Shared Memory

Extra Resources *

{}

Extra Resources available in the cluster (ex. NVIDIA GPUs)

LAUNCH

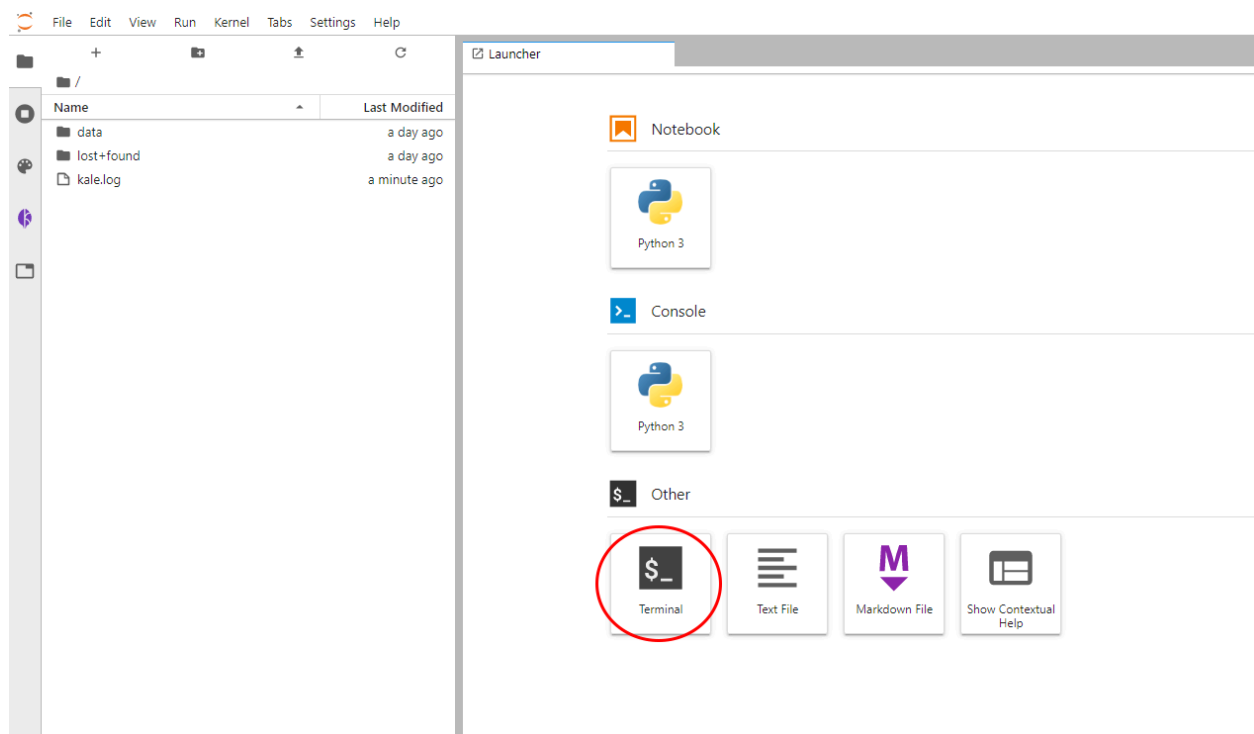
CANCEL

筆記本伺服器可用時，按一下「連線」即可連線。

Notebook Servers							+ NEW SERVER
Status	Name	Age	Image	CPU	Memory	Volumes	
✓	kubecon-tutorial	1 min ago	jupyter-kale:f20978e	0.5	1.0Gi	⋮	CONNECT 🗑️

下載資料和筆記本

系統會開啟新分頁，並顯示 JupyterLab 登陸頁面。在 JupyterLab 中建立新的終端機。

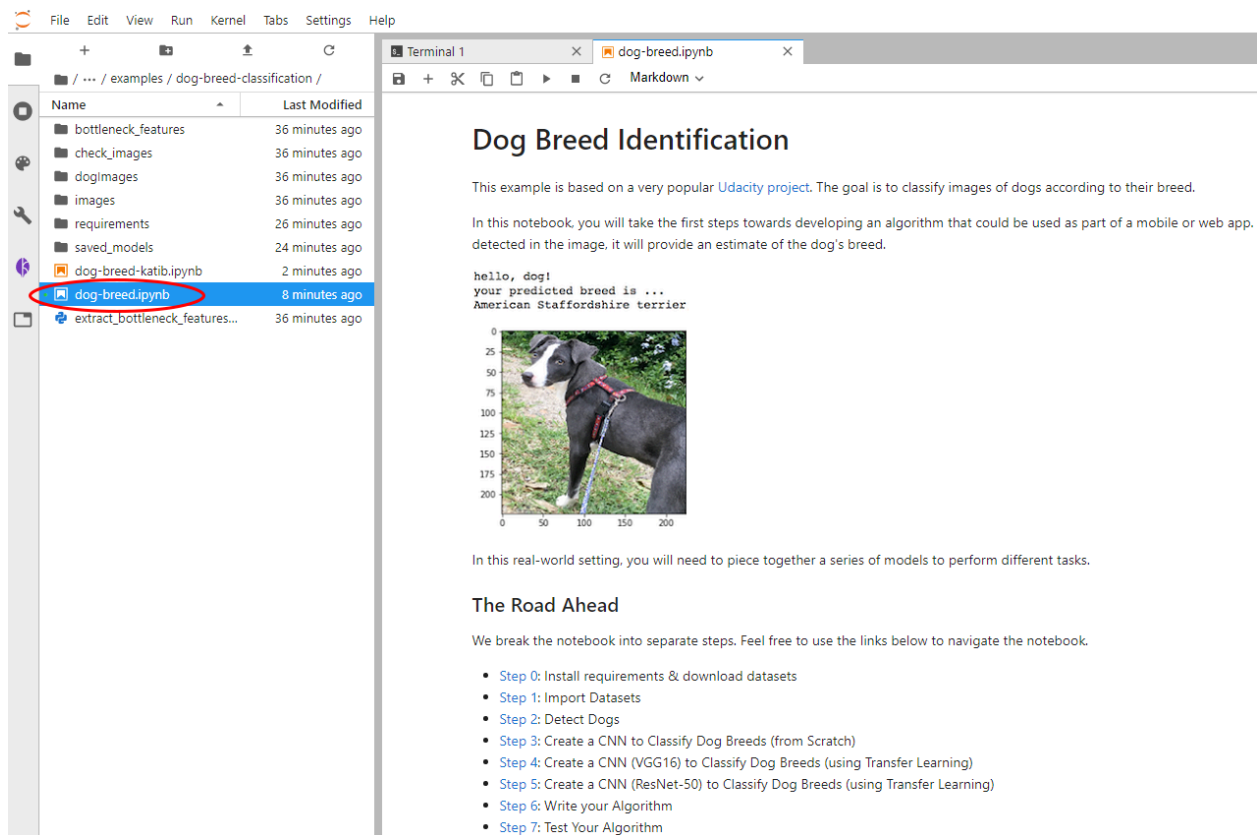


在終端機視窗中執行下列指令，前往 **data** 資料夾，並下載本實驗室其餘部分會用到的筆記本和資料：

```
cd data/  
git clone https://github.com/kubeflow-kale/kale
```

複製的存放區包含一系列精選範例，內含資料和註解筆記本。

在側欄中，前往 `data/kale/examples/dog-breed-classification/` 資料夾並開啟 `dog-breed.ipynb` 筆記本。



探索「狗品種辨識」範例的機器學習程式碼

目前請勿執行下載資料集的儲存格，因為您要使用剛複製的存放區中包含的一些較小資料集。如果您在家中自行執行這個範例，歡迎下載資料集。

執行 `imports` 儲存格，匯入所有必要程式庫。請注意，程式碼會失敗，因為缺少程式庫：

Below is the `imports` cell. This is where we import all the necessary libraries

```
[1]: from sklearn.datasets import load_files
from keras.utils import np_utils
import numpy as np
from glob import glob

import os
import random

import cv2
import matplotlib.pyplot as plt

from keras.applications.resnet50 import ResNet50

from keras.preprocessing import image
from tqdm import tqdm

from keras.applications.resnet50 import preprocess_input, decode_predictions

from PIL import ImageFile
ImageFile.LOAD_TRUNCATED_IMAGES = True

from keras.layers import Conv2D, MaxPooling2D, GlobalAveragePooling2D
from keras.layers import Dropout, Flatten, Dense
from keras.models import Sequential

from keras.callbacks import ModelCheckpoint

import extract_bottleneck_features as ebf

from keras import optimizers
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
<ipython-input-1-8049267bc436> in <module>
----> 1 from sklearn.datasets import load_files
      2 from keras.utils import np_utils
      3 import numpy as np
      4 from glob import glob
      5

ModuleNotFoundError: No module named 'sklearn'
```

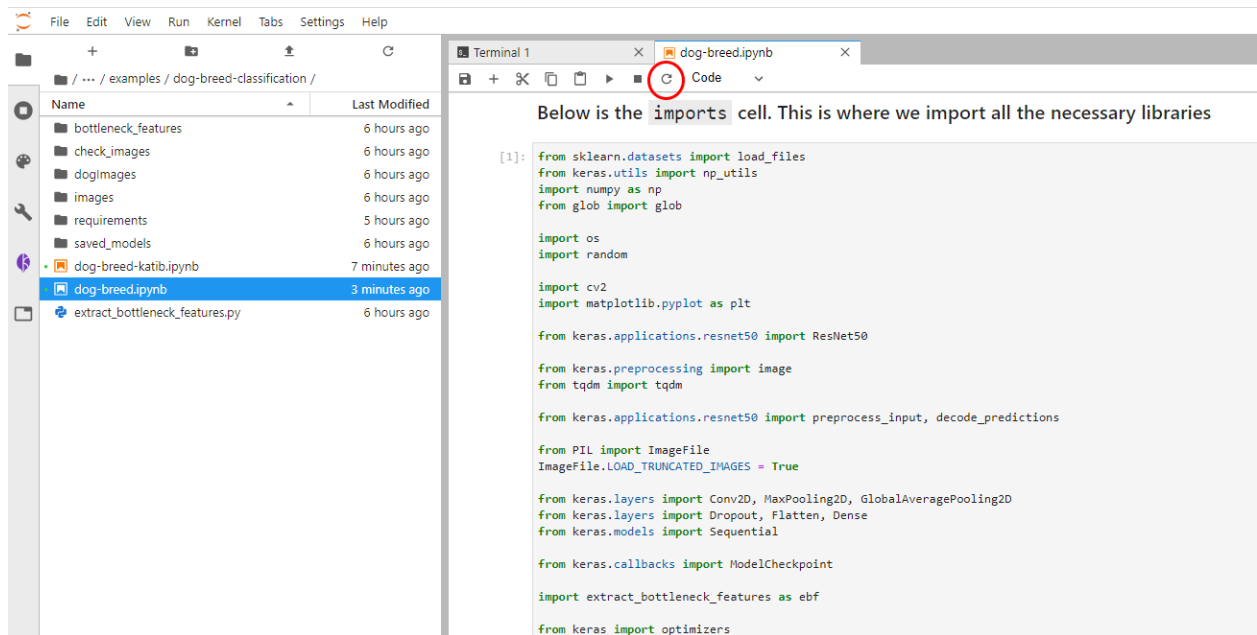
一般來說，您應該建立新的 Docker 映像檔，才能將這個 Notebook 做為 Kubeflow pipeline 執行，並納入新安裝的程式庫。幸好，Rok 和 Kale 會確保您在開發期間安裝的任何程式庫都會進入管道，這要歸功於 Rok 的快照技術，以及 Kale 將這些快照磁碟區裝載到管道步驟中。

執行下一個儲存格，安裝缺少的程式庫：

```
[2]: !pip3 install --user -r requirements/requirements.txt

Collecting opencv-python==3.2.0.6
  Downloading opencv_python-3.2.0.6-cp36-cp36m-manylinux1_x86_64.whl (6.7 MB)
    | 6.7 MB 5.5 MB/s eta 0:00:01 | 2.5 MB 5.5 MB/s eta 0:00:01
Collecting h5py==2.6.0
  Downloading h5py-2.6.0-cp36-cp36m-manylinux1_x86_64.whl (4.8 MB)
    | 4.8 MB 75.8 MB/s eta 0:00:01
Collecting matplotlib==2.0.0
  Downloading matplotlib-2.0.0-1-cp36-cp36m-manylinux1_x86_64.whl (14.7 MB)
    | 14.7 MB 41.3 MB/s eta 0:00:01
Collecting numpy==1.12.0
  Downloading numpy-1.12.0-cp36-cp36m-manylinux1_x86_64.whl (16.8 MB)
    | 16.8 MB 10.7 MB/s eta 0:00:01
Collecting scipy==0.18.1
  Downloading scipy-0.18.1-cp36-cp36m-manylinux1_x86_64.whl (42.5 MB)
    | 42.5 MB 24.6 MB/s eta 0:00:01 | 24.0 MB 24.6 MB/s eta 0:00:01 | 38.7 MB 24.6 MB/s eta 0:
00:01
Collecting tqdm==4.11.2
  Downloading tqdm-4.11.2-py2.py3-none-any.whl (46 kB)
    | 46 kB 5.6 MB/s eta 0:00:01
Collecting keras==2.0.2
  Downloading Keras-2.0.2.tar.gz (192 kB)
    | 192 kB 69.4 MB/s eta 0:00:01
Collecting scikit-learn==0.18.1
  Downloading scikit_learn-0.18.1-cp36-cp36m-manylinux1_x86_64.whl (11.8 MB)
    | 11.8 MB 46.0 MB/s eta 0:00:01
Collecting pillow==4.0.0
  Downloading Pillow-4.0.0-cp36-cp36m-manylinux1_x86_64.whl (5.6 MB)
    | 5.6 MB 61.1 MB/s eta 0:00:01
```

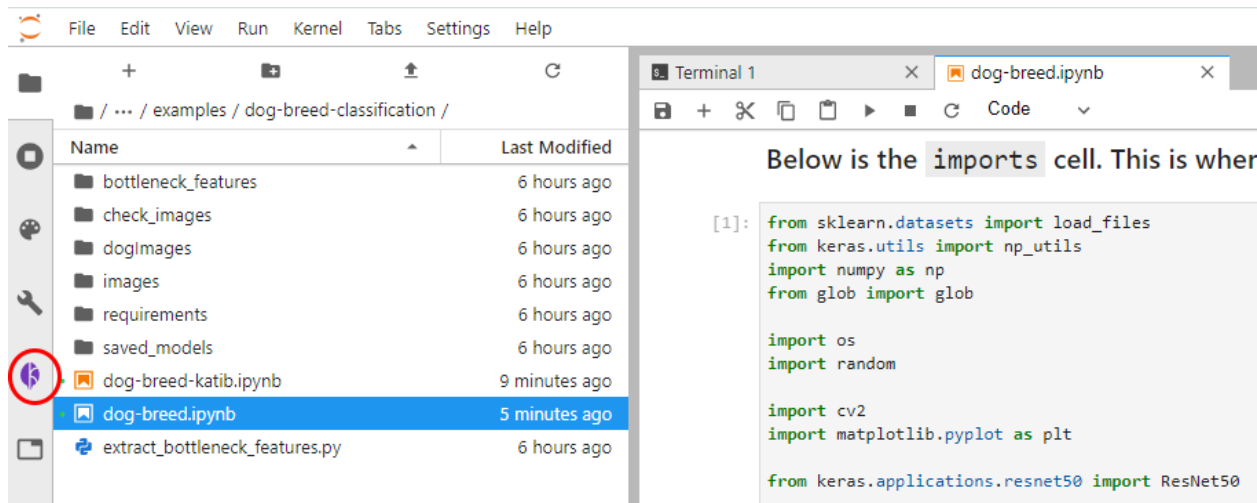
按一下「Restart」圖示，重新啟動筆記本核心：



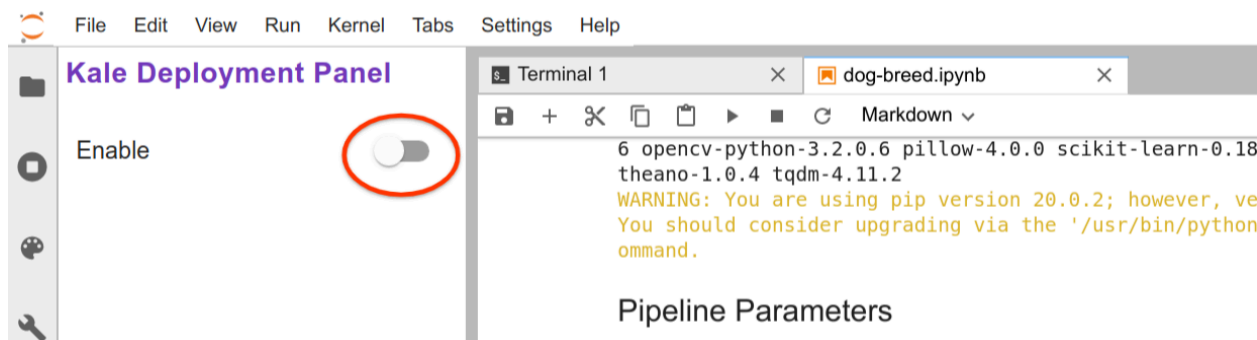
安裝正確的程式庫後，再次執行 `imports` 儲存格，即可順利完成。

在 Kubeflow Pipelines 中將筆記本轉換為管道

按一下筆記本左側窗格中的 Kubeflow 圖示，啟用 Kale：



按一下「Kale Deployment Panel」(Kale 部署面板) 中的滑桿，啟用 Kale：



探索筆記本中各儲存格的依附元件。如要瞭解多個筆記本儲存格如何成為單一管道步驟的一部分 (如儲存格左側的色條所示)，以及管道步驟如何取決於先前的步驟 (如儲存格上方的「depends on」標籤所示)，舉例來說，下圖顯示屬於相同管道步驟的多個儲存格。兩者顏色相同 (紅色)，且取決於先前的管道步驟。

Obtain Bottleneck Features

step: **cnn_vgg16** depends on: ●

Cell type
Pipeline Step

Step name
cnn_vgg16

Depends on
detect_dogs

GPU X

```
[ ]: bottleneck_features = np.load('bottleneck_features/DogVGG16Data.npz')
train_VGG16 = bottleneck_features['train']
valid_VGG16 = bottleneck_features['valid']
test_VGG16 = bottleneck_features['test']
```

Model Architecture

The model uses the the pre-trained VGG-16 architecture as a fixed feature extractor, where the last convolutional output of VGG-16 is fed as input to our model. We only add a global average pooling layer and a fully connected layer, where the latter contains one node for each dog category and is equipped with a softmax.

```
[ ]: VGG16_model = Sequential()
VGG16_model.add(GlobalAveragePooling2D(input_shape=train_VGG16.shape[1:]))
VGG16_model.add(Dense(100, activation='softmax'))
VGG16_model.summary()
```

Compile the Model

```
[ ]: VGG16_model.compile(loss='categorical_crossentropy', optimizer='rmsprop', metrics=['accuracy'])
```

按一下「Compile and Run」(編譯並執行) 按鈕：

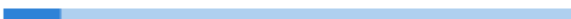
The image shows the Kale Deployment Panel interface. At the top, there is a menu bar with options: File, Edit, View, Run, Kernel, Tabs, Settings, and Help. The main panel is titled 'Kale Deployment Panel' and contains several sections:

- Enable**: A toggle switch that is currently turned on (blue).
- Pipeline Metadata**:
 - Select experiment**: A dropdown menu showing 'dog-breed-katib'.
 - Pipeline Name**: A text field showing 'dog-breed'.
 - Pipeline Description**: A text field that is currently empty.
- Run**:
 - HP Tuning with Katib**: A toggle switch that is currently turned off (grey).
 - SET UP KATIB JOB**: A button that is currently disabled (greyed out).
 - Use this notebook's volumes**: A toggle switch that is currently turned on (blue).
 - Take Rok snapshots before each step**: A toggle switch that is currently turned on (blue).
- Advanced Settings**: A section that is currently collapsed (indicated by a downward arrow).

At the bottom of the panel, there is a large blue button labeled 'COMPILE AND RUN', which is circled in red. To the right of this button is a vertical ellipsis menu icon.

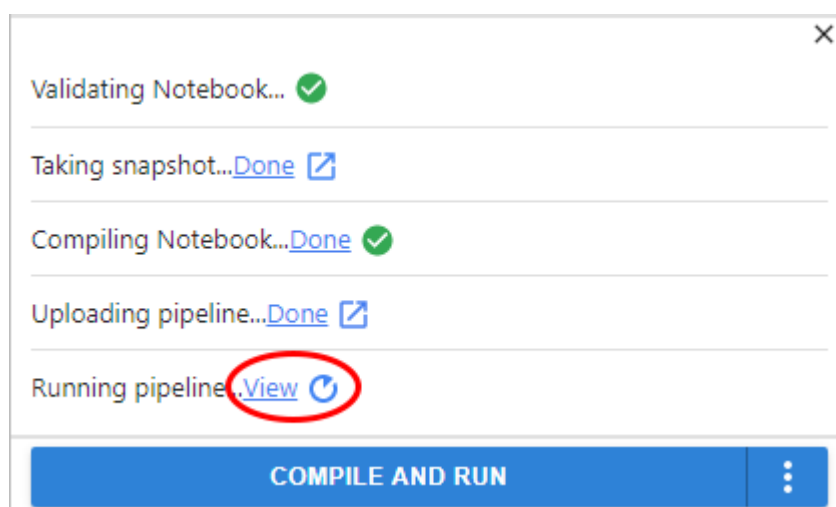
現在 Kale 會接手，將筆記本轉換為 Kubeflow Pipelines 管道，並建構筆記本。此外，由於 Kale 會與 Rok 整合，擷取目前筆記本的資料量快照，因此您可以查看快照進度。Rok 會負責處理資料版本管理，並重現您點選「Compile and Run」(編譯並執行) 按鈕時的整個環境。這樣一來，您就能擁有資料和程式碼的時間機器，且管道會在您開發程式碼的相同環境中執行，不必建構新的 Docker 映像檔。

Validating notebook... 

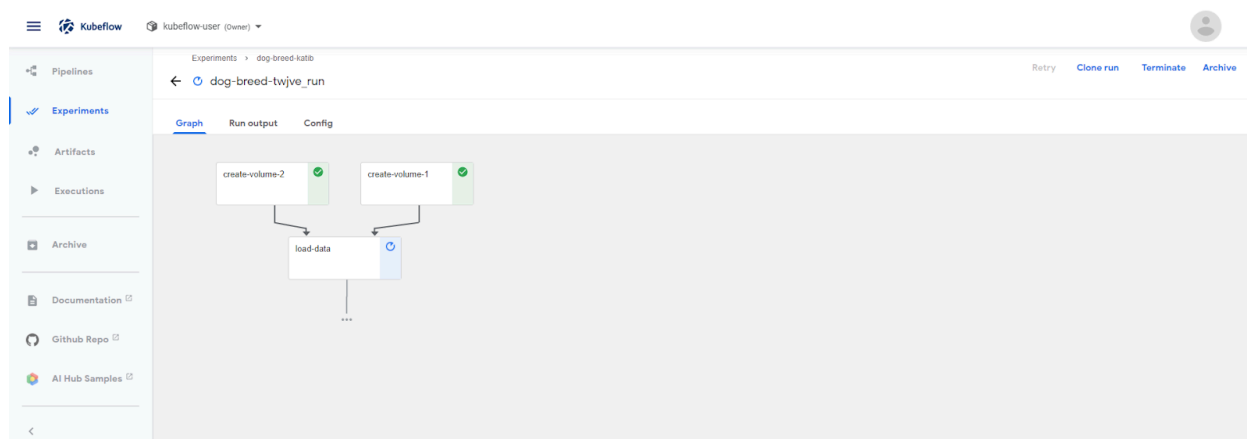
Taking snapshot... 



管道已編譯並上傳至 Kubeflow Pipelines。現在請點選連結前往 Kubeflow Pipelines UI，並查看執行作業。



Kubeflow Pipelines 使用者介面會在新分頁中開啟。等待執行完成。





恭喜！您剛才在 Kubeflow Pipelines 中執行端對端管道，一切都從筆記本開始！

5. 使用超參數調整進行遷移學習

查看結果

注意：這些結果是在使用大型資料集執行時取得。如果您使用 Kale 存放區中包含的小型資料集執行管道，這項程式碼研究室的結果會差很多。

查看 **cnn-from-scratch** 步驟的記錄。(在 Kubeflow Pipelines 使用者介面中，按一下圖表中的步驟，然後點選「記錄」分頁標籤)。這個步驟是從頭訓練卷積類神經網路 (CNN)。請注意，訓練模型的準確率非常低，而且完成這個步驟需要很長時間。

×

dog-breed-jderl-5qhw4-1512145437

ArtifactsInput/OutputVolumesManifestLogs

```
8 Loading numpy obj: test_files.npy
9 Loading numpy obj: test_targets.npy
10 Loading numpy obj: train_files.npy
11 Loading numpy obj: train_targets.npy
12 Loading numpy obj: valid_files.npy
13 Loading numpy obj: valid_targets.npy
14 Using TensorFlow backend.
15 0% | 0/6680 [00:00<?, ?it/s] 0%| | 6/6680 [00:00<01:57, 56.74it/s] 0%| | 14/6680 [00:00<01:47, 61.79it/s] 0%| | 22/6680 [00:00<01:37, 66.84it/s] 0%|
16 0% | 0/835 [00:00<?, ?it/s] 0%| | 3/835 [00:00<00:52, 15.93it/s] 1%| | 12/835 [00:00<00:39, 20.77it/s] 3%| | 22/835 [00:00<00:26, 25.61it/s] 3%|
17 0% | 0/836 [00:00<?, ?it/s] 1%| | 10/836 [00:00<00:08, 99.81it/s] 3%| | 21/836 [00:00<00:07, 102.08it/s] 4%| | 32/836 [00:00<00:06, 104.35it/s] 4%|
18
19 Layer (type) Output Shape Param #
20 -----
21 conv2d_1 (Conv2D) (None, 223, 223, 16) 208
22
23 max_pooling2d_1 (MaxPooling2D) (None, 111, 111, 16) 0
24
25 conv2d_2 (Conv2D) (None, 110, 110, 32) 2080
26
27 max_pooling2d_2 (MaxPooling2D) (None, 55, 55, 32) 0
28
29 conv2d_3 (Conv2D) (None, 54, 54, 64) 8256
30
31 max_pooling2d_3 (MaxPooling2D) (None, 27, 27, 64) 0
32
33 global_average_pooling2d_1 (GlobalAveragePooling2D) (None, 64) 0
34
35 dense_1 (Dense) (None, 133) 8645
36 -----
37 Total params: 19,189.0
38 Trainable params: 19,189.0
39 Non-trainable params: 0.0
40
41 W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use SSE3 instructions, but these are available on your machine
42 W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use SSE4.1 instructions, but these are available on your machine
43 W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use SSE4.2 instructions, but these are available on your machine
44 W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use AVX instructions, but these are available on your machine
45 W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use AVX2 instructions, but these are available on your machine
46 W tensorflow/core/platform/cpu_feature_guard.cc:45] The TensorFlow library wasn't compiled to use FMA instructions, but these are available on your machine
47 Train on 6680 samples, validate on 835 samples
48 Epoch 1/1
49 32/6680 [.....] - ETA: 251s - loss: 4.8981 - acc: 0.0000e+00
50 6680/6680 [=====] - 254s - loss: 4.8981 - acc: 0.0000e+00
51 Test accuracy: 1.4354%
```

查看 **cnn-vgg16** 步驟的記錄。在這個步驟中，您對預先訓練的 VGG-16 模型 (由 Visual Geometry Group (VGG) 訓練的類神經網路) 進行遷移學習。準確率比先前的模型高出許多，但仍有進步空間。

dog-breed-jderl-5qhw4-3782927250

[illegible]

現在，請查看 **cnn-resnet50** 步驟的記錄。在本步驟中，您對預先訓練的 ResNet-50 模型使用了遷移學習。準確度大幅提升。因此，您會在接下來的程式碼研究室中使用這個模型。

dog-breed-jderl-5qhw4-1290017143

[illegible]

超參數調整

返回 KubeFlow UI 中的筆記本伺服器，然後開啟名為 `dog-breed-katib.ipynb` 的筆記本 (位於 `data/kale/examples/dog-breed-classification/` 路徑)。您將使用 Katib，對這個筆記本中的 ResNet-50 模型執行一些超參數調整實驗。請注意，筆記本開頭有一個儲存格，用於宣告參數：

pipeline-parameters

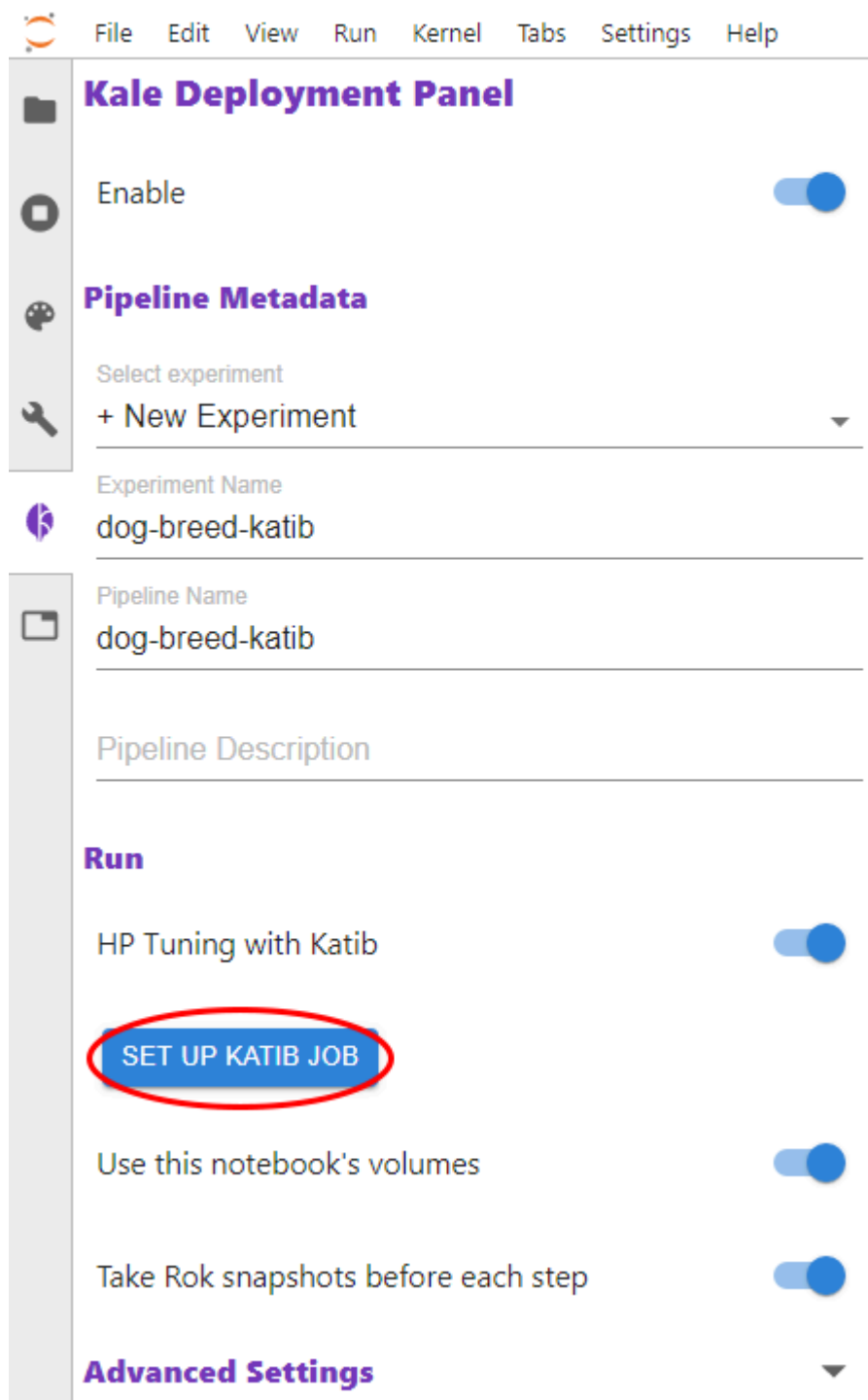
```
[3]: nodes_number = 256  
     learning_rate = 0.0001
```

在筆記本的左側窗格中，啟用「HP Tuning with Katib」，執行超參數調整：

The screenshot shows the 'Kale Deployment Panel' interface. At the top is a menu bar with 'File', 'Edit', 'View', 'Run', 'Kernel', 'Tabs', 'Settings', and 'Help'. The left sidebar contains icons for a folder, a circle with a square, a palette, a wrench, and a document. The main panel has the following sections:

- Kale Deployment Panel**
 - Enable**: A toggle switch that is currently turned on (blue).
- Pipeline Metadata**
 - Select experiment**: A dropdown menu showing '+ New Experiment'.
 - Experiment Name**: A text field containing 'dog-breed-katib'.
 - Pipeline Name**: A text field containing 'dog-breed-katib'.
 - Pipeline Description**: An empty text field.
- Run**
 - HP Tuning with Katib**: A toggle switch that is currently turned off (grey). This toggle is circled in red.
 - SET UP KATIB JOB**: A grey button.
 - Use this notebook's volumes**: A toggle switch that is currently turned on (blue).
 - Take Rok snapshots before each step**: A toggle switch that is currently turned on (blue).
- Advanced Settings**: A section header with a downward arrow.

然後按一下「Set up Katib Job」(設定 Katib 工作)，設定 Katib：



The screenshot shows the 'Kale Deployment Panel' with a sidebar on the left containing icons for a folder, a square, a palette, a wrench, and a document. The main panel has a menu bar with 'File', 'Edit', 'View', 'Run', 'Kernel', 'Tabs', 'Settings', and 'Help'. The 'Kale Deployment Panel' title is in purple. Below it, the 'Enable' toggle is turned on. The 'Pipeline Metadata' section includes a 'Select experiment' dropdown menu with '+ New Experiment' and a dropdown arrow. Below this, the 'Experiment Name' is 'dog-breed-katib'. The 'Pipeline Name' is also 'dog-breed-katib'. The 'Pipeline Description' field is empty. The 'Run' section has three toggles: 'HP Tuning with Katib' (on), 'SET UP KATIB JOB' (highlighted with a red oval), and 'Use this notebook's volumes' (on). The 'Take Rok snapshots before each step' toggle is also on. The 'Advanced Settings' section is collapsed with a dropdown arrow.

Kale Deployment Panel

Enable ☒

Pipeline Metadata

Select experiment
+ New Experiment ▼

Experiment Name
dog-breed-katib

Pipeline Name
dog-breed-katib

Pipeline Description

Run

HP Tuning with Katib ☒

SET UP KATIB JOB

Use this notebook's volumes ☒

Take Rok snapshots before each step ☒

Advanced Settings ▼

為每個參數定義搜尋空間，並定義目標：

Katib Job

Search Space Parameters ?

Name: **nodes_number**
Type: **int**
☒ Range

Min: 256 Max: 1200 Step: 256

Name: **learning_rate**
Type: **double**
☒ List

Value: 0.001 

Value: 0.005 

Value: 0.0001 

Value: 0.0005 

[+ ADD VALUE](#)

Search Algorithm ?

Algorithm: Grid Search

Search Objective ?

Reference Metric: test_accuracy_...
Type: Maximize
Goal: 90

Run Parameters ?

Parallel Trial Count: 1
Max Trial Count: 200
max Failed Trial Count: 3

[CLOSE](#)

按一下「Compile and Run Katib Job」(編譯並執行 Katib 工作) 按鈕：

Run

HP Tuning with Katib



SET UP KATIB JOB

Use this notebook's volumes



Take Rok snapshots before each step



Advanced Settings



COMPILE AND RUN KATIB JOB



查看 Katib 實驗的進度：

×

Validating notebook... 

Taking snapshot... [Done](#) 

Compiling notebook... [Done](#) 

Uploading pipeline... [Done](#) 

Creating KFP experiment... [Done](#) 

Running Katib experiment... [View](#) 

Gathering suggestions...

COMPILE AND RUN KATIB JOB



按一下「查看」即可查看 Katib 實驗：



Validating notebook... 

Taking snapshot... [Done](#) 

Compiling notebook... [Done](#) 

Uploading pipeline... [Done](#) 

Creating KFP experiment... [Done](#) 

Running Katib experiment... [View](#) 

Running: 1

Succeeded: 0

Failed: 0

COMPILE AND RUN KATIB JOB



按一下「完成」，即可在 Kubeflow Pipelines (KFP) 中查看執行作業：

×

Validating notebook...

Taking snapshot... [Done](#)

Compiling notebook... [Done](#)

Uploading pipeline... [Done](#)

Creating KFP experiment... [Done](#)

Running Katib experiment... [View](#)

Running: 1
Succeeded: 0
Failed: 0

COMPILE AND RUN KATIB JOB

⋮

在 Katib 實驗頁面中，您會看到新的試驗：

≡ Katib

BACK

Experiment Name: dog-breed-katib-uvftj
Experiment Namespace: kubeflow-user
VIEW EXPERIMENT

trialName	Status	test-accuracy-resnet	nodes_number	learning_rate
dog-breed-katib-uvftj-6487aa2c	Running		256	0.001

在 KFP UI 中，您會看到新的執行作業：

Runs

+ Create run

+ Create recurring run

Compare runs

Clone run

Archive

Filter runs

☐ Run name	Status	Duration	Pipeline Version	Recurring Run	Start time ↓
☐ dog-breed-katib-uvftj-6487aa2c		-	dog-breed-g1s0n	-	06/05/2020, 16:18:53

Rows per page: 10 < >

我們來看看剛剛發生了什麼事。先前，Kale 是從筆記本產生管道執行作業，現在則會產生多個管道執行作業，每個作業都會饋送不同的引數組合。

Katib 是 Kubeflow 的元件，可執行一般用途的超參數調整工作。Katib 對實際執行的工作 (在 Katib 術語中稱為試驗) 一無所知。Kale 只會關注搜尋空間、最佳化演算法和目標。Katib 支援以試驗形式執行簡單的 Job (即 Pod)，但 Kale 會實作墊片，讓試驗實際在 Kubeflow Pipelines 中執行管道，然後從管道執行作業收集指標。

Katib 實驗產生試驗時，您會在 Katib UI 中看到更多試驗：

trialName	Status	test-accuracy-resnet	nodes_number	learning_rate
dog-breed-katib-uvftj-20d6fbad	Running		512	0.001
dog-breed-katib-uvftj-21a750c9	Succeeded	83.01435406698565	256	0.0001
dog-breed-katib-uvftj-6487aa2c	Succeeded	82.29665071770334	256	0.001
dog-breed-katib-uvftj-66956233	Succeeded	67.82296650717703	256	0.005
dog-breed-katib-uvftj-821387e1	Succeeded	83.49282296650718	256	0.0005

在 KFP UI 中執行更多作業：

Runs

+ Create run

+ Create recurring run

Compare runs

Clone run

Archive

Filter runs

☐	Run name	Status	Duration	Pipeline Version	Recurring Run...	Start time ↓	test-accuracy-...
☐	dog-breed-katib-uvftj-20d6fbad		-	dog-breed-g1s0n	-	06/05/2020, 16:42:11	
☐	dog-breed-katib-uvftj-821387e1		0:05:11	dog-breed-g1s0n	-	06/05/2020, 16:36:30	83.493
☐	dog-breed-katib-uvftj-21a750c9		0:05:01	dog-breed-g1s0n	-	06/05/2020, 16:30:47	83.014
☐	dog-breed-katib-uvftj-66956233		0:05:40	dog-breed-g1s0n	-	06/05/2020, 16:24:35	67.823
☐	dog-breed-katib-uvftj-6487aa2c		0:05:18	dog-breed-g1s0n	-	06/05/2020, 16:18:53	82.297

Rows per page: 10 < >

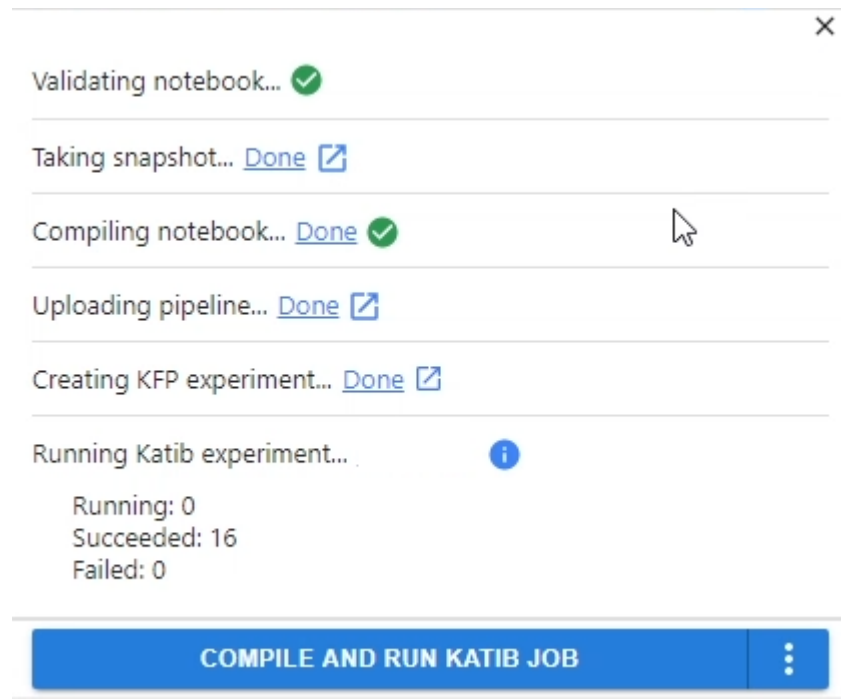
Katib 實驗完成後，您可以在 Katib UI 中查看所有試驗：

trialName	Status	test-accuracy-resnet	nodes_number	learning_rate
dog-breed-katib-uvftj-030c20dc	Succeeded	71.88995215311004	768	0.005
dog-breed-katib-uvftj-10a5637b	Succeeded	82.53588516746412	768	0.001
dog-breed-katib-uvftj-20d6fbad	Succeeded	81.69856459330144	512	0.001
dog-breed-katib-uvftj-21a750c9	Succeeded	83.01435406698565	256	0.0001
dog-breed-katib-uvftj-623a5c8b	Succeeded	81.22009569377991	1024	0.0005
dog-breed-katib-uvftj-6345e894	Succeeded	83.13397129186603	1024	0.0001
dog-breed-katib-uvftj-639e513e	Succeeded	70.33492822966507	1024	0.005
dog-breed-katib-uvftj-6487aa2c	Succeeded	82.29665071770334	256	0.001
dog-breed-katib-uvftj-66956233	Succeeded	67.82296650717703	256	0.005
dog-breed-katib-uvftj-6dd91746	Succeeded	70.93301435406698	512	0.005
dog-breed-katib-uvftj-80a78352	Succeeded	83.73205741626795	768	0.0001
dog-breed-katib-uvftj-821387e1	Succeeded	83.49282296650718	256	0.0005
dog-breed-katib-uvftj-84345cf1	Succeeded	82.29665071770334	512	0.0005
dog-breed-katib-uvftj-8937a9ae	Succeeded	84.0909090909091	512	0.0001
dog-breed-katib-uvftj-c619218f	Succeeded	81.22009569377991	1024	0.001
dog-breed-katib-uvftj-ca0aef2e	Succeeded	83.13397129186603	768	0.0005

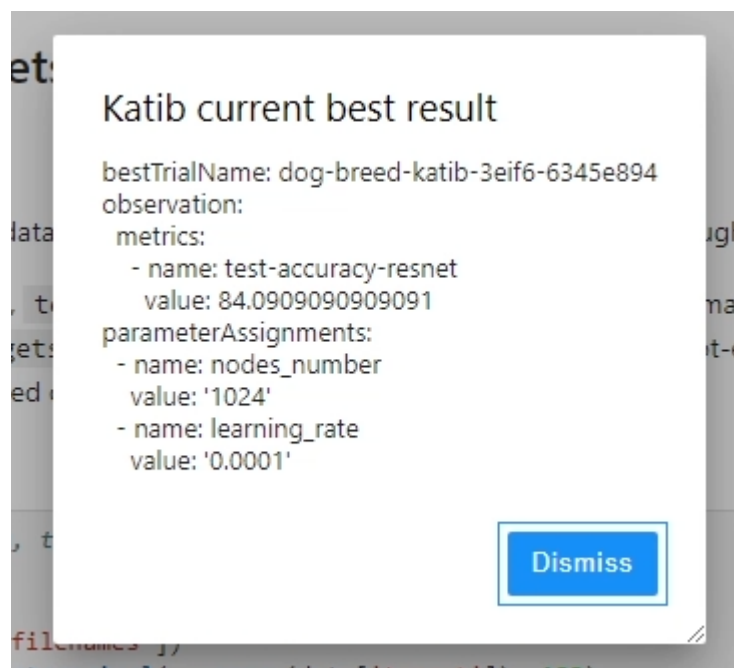
以及 KFP UI 中的所有執行作業：

Runs							
+ Create run + Create recurring run Compare runs Clone run Archive Filter runs							
☐	Run name	Status	Duration	Pipeline Version	Recurring Run...	Start time ↓	test-accuracy-...
☐	dog-breed-katib-uvftj-623a5c8b	✓	0:08:07	dog-breed-g1s0n	-	06/05/2020, 18:08:26	81.220
☐	dog-breed-katib-uvftj-6345e894	✓	0:07:45	dog-breed-g1s0n	-	06/05/2020, 18:00:12	83.134
☐	dog-breed-katib-uvftj-639e513e	✓	0:08:39	dog-breed-g1s0n	-	06/05/2020, 17:51:00	70.335
☐	dog-breed-katib-uvftj-c619218f	✓	0:08:24	dog-breed-g1s0n	-	06/05/2020, 17:41:48	81.220
☐	dog-breed-katib-uvftj-ca0aef2e	✓	0:06:55	dog-breed-g1s0n	-	06/05/2020, 17:34:06	83.134
☐	dog-breed-katib-uvftj-80a78352	✓	0:06:50	dog-breed-g1s0n	-	06/05/2020, 17:26:54	83.732
☐	dog-breed-katib-uvftj-030c20dc	✓	0:07:44	dog-breed-g1s0n	-	06/05/2020, 17:18:41	71.890
☐	dog-breed-katib-uvftj-10a5637b	✓	0:07:37	dog-breed-g1s0n	-	06/05/2020, 17:10:29	82.536
☐	dog-breed-katib-uvftj-84345cf1	✓	0:06:19	dog-breed-g1s0n	-	06/05/2020, 17:03:47	82.297
☐	dog-breed-katib-uvftj-8937a9ae	✓	0:06:16	dog-breed-g1s0n	-	06/05/2020, 16:57:05	84.091
☐	dog-breed-katib-uvftj-6dd91746	✓	0:07:05	dog-breed-g1s0n	-	06/05/2020, 16:49:23	70.933
☐	dog-breed-katib-uvftj-20d6fbad	✓	0:06:37	dog-breed-g1s0n	-	06/05/2020, 16:42:11	81.699
☐	dog-breed-katib-uvftj-821387e1	✓	0:05:11	dog-breed-g1s0n	-	06/05/2020, 16:36:30	83.493
☐	dog-breed-katib-uvftj-21a750c9	✓	0:05:01	dog-breed-g1s0n	-	06/05/2020, 16:30:47	83.014
☐	dog-breed-katib-uvftj-66956233	✓	0:05:40	dog-breed-g1s0n	-	06/05/2020, 16:24:35	67.823
☐	dog-breed-katib-uvftj-6487aa2c	✓	0:05:18	dog-breed-g1s0n	-	06/05/2020, 16:18:53	82.297

返回 Notebook 後，您會在 Kale 面板中看到 Katib 實驗旁的資訊按鈕：



按一下即可查看最佳結果和產生該結果的參數：



步驟 6 · 約 5 分鐘

6. 清除所用資源

終止 MiniKF VM

前往 GCP Console 的 [Deployment Manager](#)，然後刪除 `minikf-on-gcp` 部署作業。

步驟 7 · 約 0 分鐘

7. 恭喜

恭喜！您已成功使用 Kubeflow (MiniKF)、Kale 和 Rok，執行端對端資料科學工作流程！

後續步驟

加入 Kubeflow 社群：

- github.com/kubeflow
- [Kubeflow Slack](#)
- kubeflow-discuss@googlegroups.com
- [每週社群通話](#)、[Slack](#) 和其他社群詳細資料

其他資訊

- [Kubeflow 說明文件](#)
 - [Medium 上的 Kale 簡介](#)
 - [Kale GitHub 組織](#)
 - [從 Notebook 透過 MiniKF 和 Kale 遷移至 Kubeflow Pipelines](#)
 - [GCP 上的 MiniKF](#)
 - [筆電上的 MiniKF](#)
-